UW-Madison GI Tract Image Segmentation

Track Healthy Organs in Medical Scans to Improve Cancer Treatment

William Ohonba Sofia Rojas Peyton Wade

Florida State University

Spring 2026

Agenda

1. Project Recap
2. Dataset Overview
3. DataLoader Outline
4. Preprocessing Pipeline
5. Augmentation Plans
6. Evaluation Metrics
7. Next Steps
8. Conclusion

Project Recap

The Challenge

This challenge hosted by the **UW-Madison Carbone Cancer Center**, aims to **automate the segmentation of the stomach and bowels** in MRI scans to enhance the precision of cancer radiotherapy.

- **Task:** Pixel-wise semantic segmentation of GI tract organs.
- **Output:** Binary masks converted to **Run-Length Encoding (RLE)** for efficiency.

The Importance

Enable **real-time adaptive radiotherapy**, allowing clinicians to deliver higher radiation doses to tumors while sparing healthy moving organs, ultimately reducing patient treatment time and toxic side effects.

Dataset Anatomy: Scale & Structure

Attribute	**Specification**
Total Images	38,496 MRI Slices
Case Studies	85 Patients (Patient Data Spans Multiple Days)
File Format	16-bit Grayscale PNG
Variable Resolution	e.g., 266×320 to 360×310 px
Total Size	2.47 GB

Table: Also Contains a csv of all IDs and Masks of training objects

DataLoader Outline

1. Initialization

```python
#initialization
def __init__(self, csvPath, folderPath, imgSize = 320, statsPath = None):
    self.df = pd.read_csv(csvPath) #train.csv
    self.folderPath = Path(folderPath) #train ~ the folder
    self.imgSize = imgSize #resolution size ~ can be changed, but defaulted to 320
```

- Loads `train.csv` via Pandas
- Maps the `/research` Directory

2. Path Navigation

```python
#fetches, processes, packages
def __getitem__(self, idx):
    row = self.df.iloc[idx] #grabs specific csv row

    #path navigation
    parts = row['id'].split('_') #splits string id
    caseID, dayID, sliceID = parts[0], parts[1], parts[3]
    folderPath = self.folderPath / caseID / f"{caseID}_{dayID}" / "scans"
    imgPath = list(folderPath.glob(f"slice_{sliceID}_*.png"))[0]
```

- Parses ID: Case/Day/Slice
- Dynamic `glob` path assembly

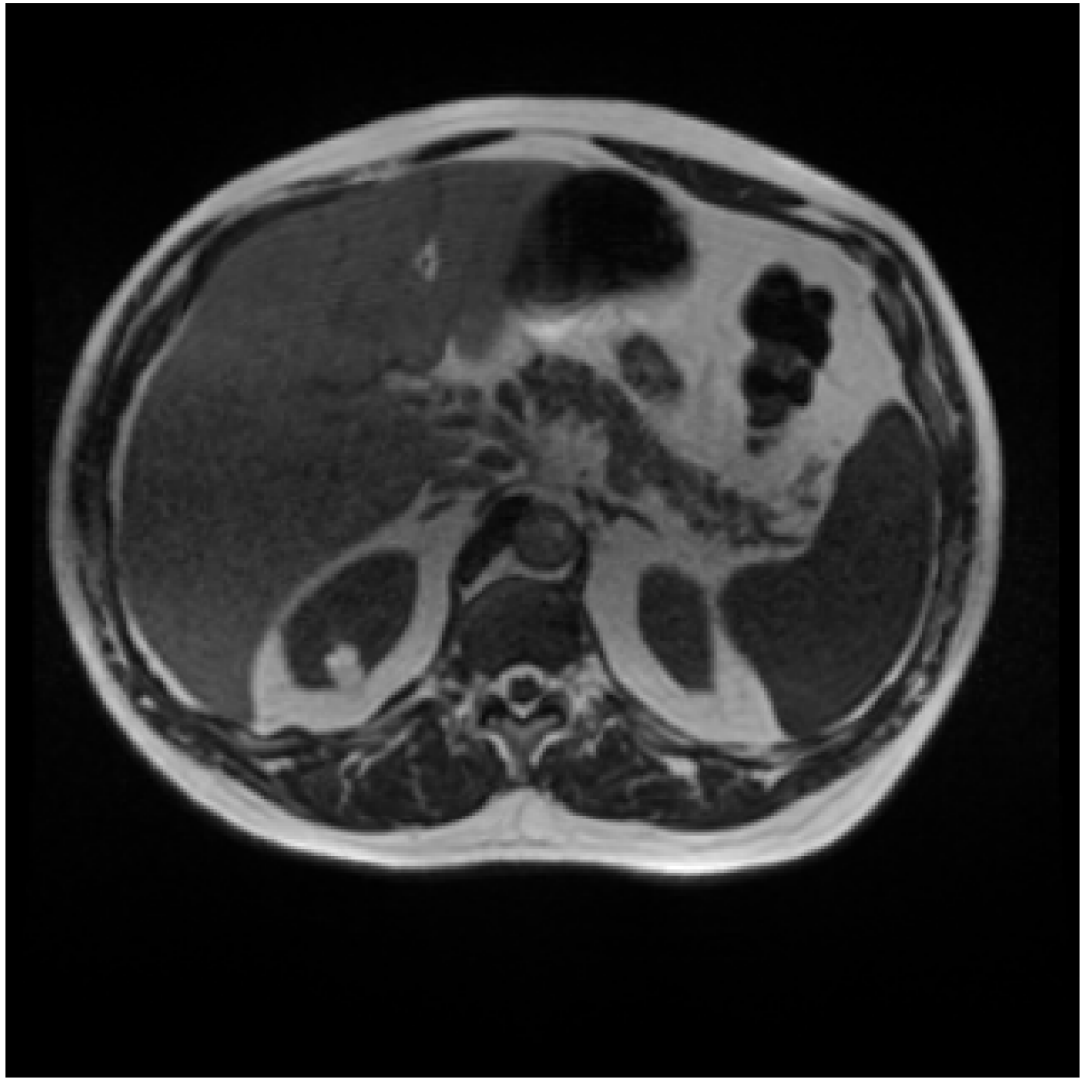
3. PyTorch DataLoader

```python
#training pass
trainDataset = GITractDataset(csvPath, folderPath, imgSize = 320, statsPath = "calcStats.json")
trainLoader = DataLoader(
    trainDataset,
    batch_size = 16,
    shuffle = True,
    num_workers = 4,
    pin_memory = True
)
```

- Batch size of 16 across 4 Workers
- Shuffle & Pin Memory enabled

DataLoader Verification

Raw MRI Scan (Slice 248)



Preprocessing Pipeline

1. Global Dataset Calibration

```python
#calibration
def calcStats(loader, savePath = "calcStats.json"):
    psum, psum_sq = 0, 0 #store sums
    count = 0 #store count

    #gather variables through data
    for imgs, _ in loader:
        psum += imgs.sum()
        psum_sq += (imgs**2).sum()
        count += imgs.numel()

    #maths
    mean = psum / count
    var  = (psum_sq / count) - (mean**2)
    std  = torch.sqrt(var)

    #stores data
    stats = {"mean": mean.item(), "std": std.item()}
    with open(savePath, "w") as f:
        json.dump(stats, f)

    return stats
```

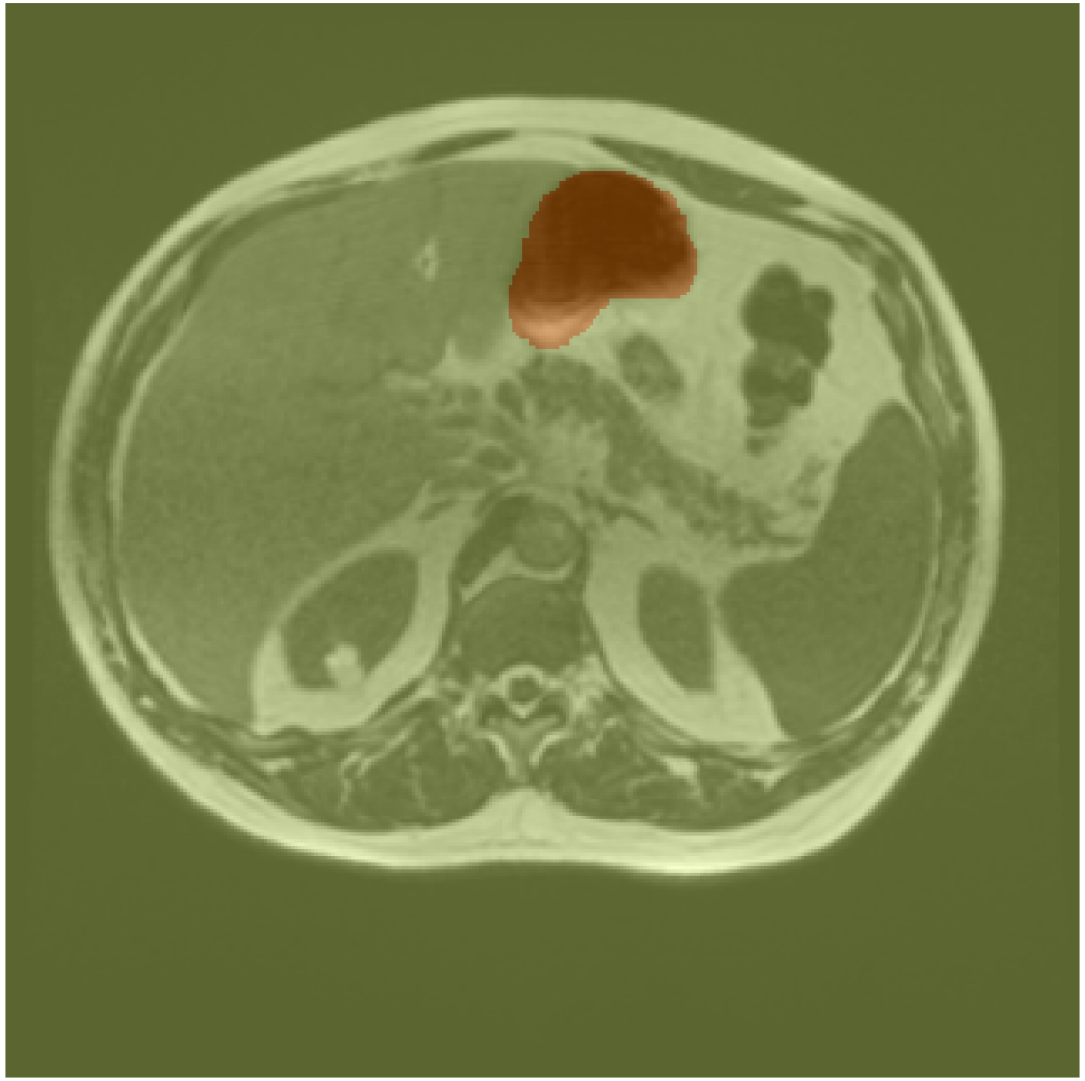
2. Scaling & Normalization

```python
#loading/preprocessing
img = cv2.imread(str(imgPath), cv2.IMREAD_UNCHANGED).astype(np.float32) #convert 16-bit to decimal
mask = self.rleDecoder(row['segmentation'], img.shape) #getting mask

#resizing
img = cv2.resize(img, (self.imgSize, self.imgSize))
mask = cv2.resize(mask, (self.imgSize, self.imgSize), interpolation=cv2.INTER_NEAREST)

#scale img between 0.0 to 1.0 ~ ensures model gradients remain stable
if img.max() > 0:
    img = img / img.max()

#normalization
img = (img - self.mean) / self.std
```

- Standardizes input to mean $= 0$ and standard deviation $= 1$ across the dataset
- Stores stats in `calcStats.json` for consistent normalization and de-normalization
- Resizing images to a uniform resolution

Preprocessing Verification

Normalized Scan with Segmentation Mask



Augmentation Plans

1. Intensity Histogram Scaling

- **Technique:** Contrast Limited Adaptive Histogram Equalization (CLAHE).
- **Goal:** Normalize varying signal intensities across different MRI scanners.

2. ROI Patch-Based Training

- **Technique:** Random cropping focused on regions with non-zero masks.
- **Goal:** AI to learn the texture of the intestines rather than just memorizing where they are in the frame.

Top Performance

Random Cropping: Scaled image sizes to 640*640 and then used random crop to target a 448*448 resolution. **Elastic Transform/Grid Distortion/Optical Distortion:** Used these series of methods to cause warping to identify organs even if stretched.

Evaluation Metrics: Overlap and Boundary Precision

1. Dice Coefficient (F1-Score)

$$DSC = \frac{2|X \cap Y|}{|X| + |Y|}$$

- **Focus:** Measures the spatial overlap between the predicted mask (X) and ground truth (Y).
- **Justification:** Ideal for imbalanced data; it ignores true negatives (background) to focus purely on the GI tract pixels.

2. 3D Hausdorff Distance (HD)

$$d_H(X, Y) = \max\{\vec{d}_H(X, Y), \vec{d}_H(Y, X)\}$$

- **Focus:** Measures the maximum distance between the surface of the prediction and the ground truth.
- **Justification:** Crucial for clinical safety; it penalizes "outlier" pixels that are far from the actual organ boundary.

Future Work: Modeling Optimization

1. Architectural Evolution

- **U-Net Baseline:** Implement a standard 2D U-Net for initial segmentation.
- **Transition to 2.5D:** Stack adjacent slices (e.g., 3-5) as input channels to provide the model with "pseudo-3D" spatial context.

2. Data Enhancements

- **Augmentation:** Integrate random ROI cropping and CLAHE.

3. Performance Evaluation

- **Validation:** Utilize `calcStats.json` to standardize validation set to simulate real-world model deployment.
- **Metric Tracking:** Compare Dice and Hausdorff scores across different model iterations.

Note: 2.5D captures MRI depth context without the massive overhead of 3D convolutions.

Questions & Discussion